

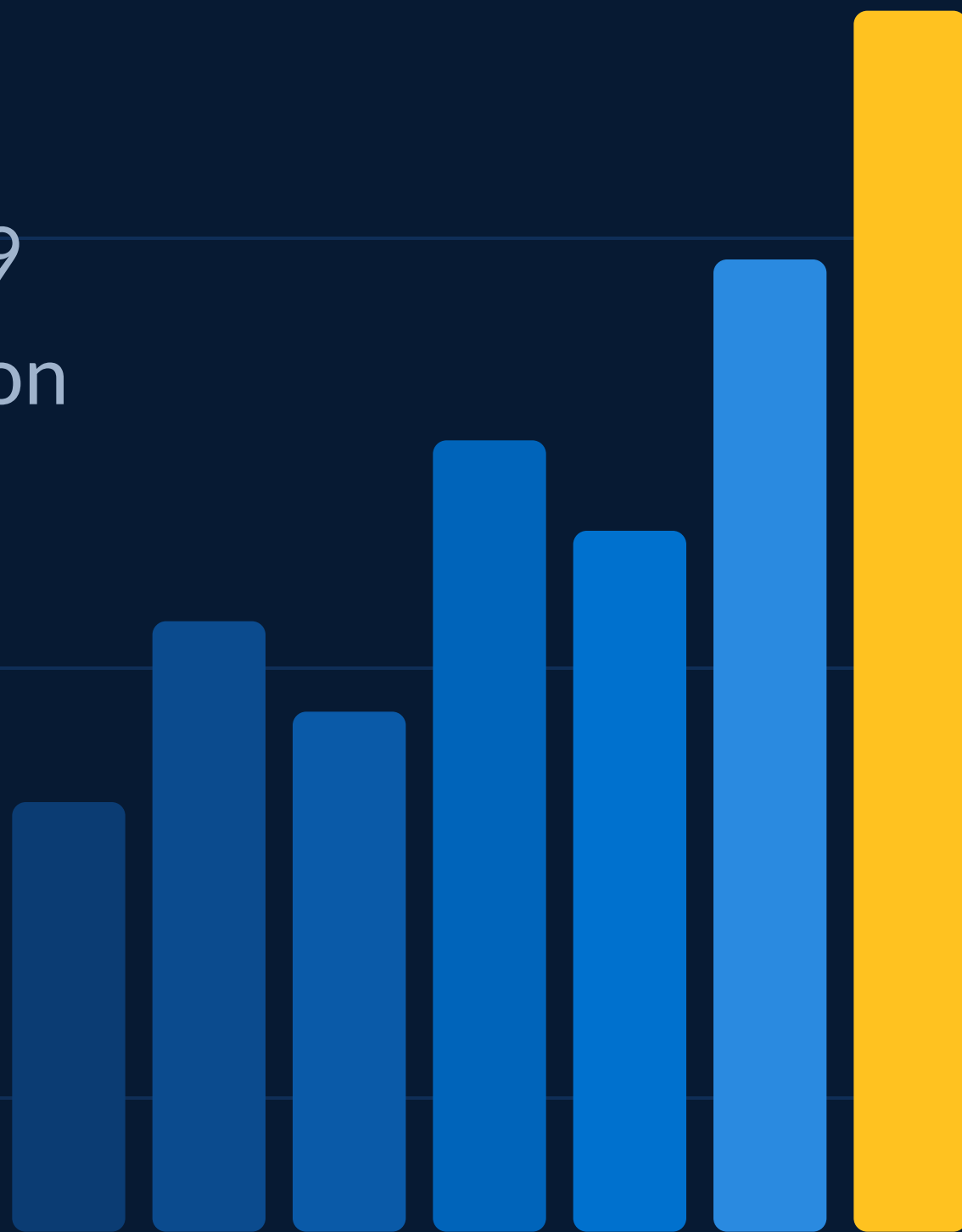


Data analytics project

Walmart Sales Analysis

From 10,051 raw invoices to 9
business answers, using Python
and SQL.

Swipe to see how it was built



The dataset

Invoice-level sales from Walmart branches. One row per transaction.

10,051

raw rows

11

columns

51

duplicate rows

31

rows with missing values

Where and who

invoice_id

Branch

City

What was sold

category

unit_price

quantity

When

date

time

How it went

payment_method

rating

profit_margin

The pipeline

1

Load

Read Walmart.csv into a pandas DataFrame.

2

Clean and transform

Remove duplicates and nulls, fix the price column, add a total column.

3

Connect

SQLAlchemy and the MySQL connector link Jupyter to the local database.

4

Write to MySQL

9,969 clean rows loaded into the walmart_sales table with to_sql.

5

Analyse in SQL

Nine business questions answered with joins, CTEs and window functions.

Cleaning the data

10,051

rows loaded

10,000

after dropping duplicates

-51

9,969

after dropping missing values

-31

cleaning.ipynb

```
df.drop_duplicates(inplace=True)
df.dropna(inplace=True)
df['unit_price'] = df['unit_price'].str.replace('$', '')
df['unit_price'] = df['unit_price'].astype(float)
df['total'] = df['unit_price'] * df['quantity']
```


Nine business questions

Q1

Payment methods:
transactions and
quantity sold

Q2

Highest-rated
category in each
branch

Q3

Busiest day for each
branch

Q4

Quantity sold per
payment method

Q5

Average, min and
max rating by city
and category

Q6

Total profit for each
category

Q7

Most common
payment method
per branch

Q8

Sales split into
morning, afternoon,
evening

Q9

Five branches with
the biggest revenue
drop, 2022 to 2023

Window functions: the winner in every group

Q2: top category per branch

```
SELECT branch, category, avg_rating
FROM (
    SELECT branch, category,
           AVG(rating) AS avg_rating,
           RANK() OVER (
               PARTITION BY branch
               ORDER BY AVG(rating) DESC
           ) AS rnk
    FROM walmart_sales
    GROUP BY branch, category
) AS ranked
WHERE rnk = 1;
```

PARTITION BY restarts the ranking inside each branch.

Wrapping it in a subquery lets WHERE rnk = 1 keep only the winner.

The same pattern answers Q3 and Q7.

CASE logic: turning clock time into shifts

Q8: invoices by shift

```
SELECT branch,  
       CASE  
         WHEN HOUR(TIME(time)) < 12 THEN 'morning'  
         WHEN HOUR(TIME(time)) BETWEEN 12 AND 17  
           THEN 'afternoon'  
         ELSE 'evening'  
       END AS shift,  
       COUNT(*) AS num_invoices  
FROM walmart_sales  
GROUP BY branch, shift  
ORDER BY branch, num_invoices DESC;
```

| HOUR() extracts the hour from each transaction time.

| CASE buckets every invoice into one of three shifts.

| Sorting by invoice count shows each branch's busiest shift first.

Profit and year-over-year change in SQL

Q6 Total profit per category

```
SUM(profit_margin * unit_price * quantity)
```

Grouped by category, so every product line gets one profit figure.

Q9 Revenue decrease ratio

```
(rev_2022 - rev_2023) / rev_2022 * 100
```

Two CTEs total revenue per branch for each year. A join on branch, a filter for declines and LIMIT 5 return the biggest drops.

Revenue itself comes from the total column added during cleaning.

Toolkit

Python side

- pandas for cleaning and transformation
- SQLAlchemy and mysql-connector for loading
- Jupyter Notebook for the workflow

SQL side

- Window functions: RANK, PARTITION BY
- CTEs for readable multi-step logic
- CASE, aggregation and date functions

Which question would you ask this data?

Tell me in the comments, or point out a better way to write any of these queries.

Built with Python, MySQL and SQL

10,051 raw rows > 9,969 clean rows > 9 answers